Holly Pasquinelli
Lab 6

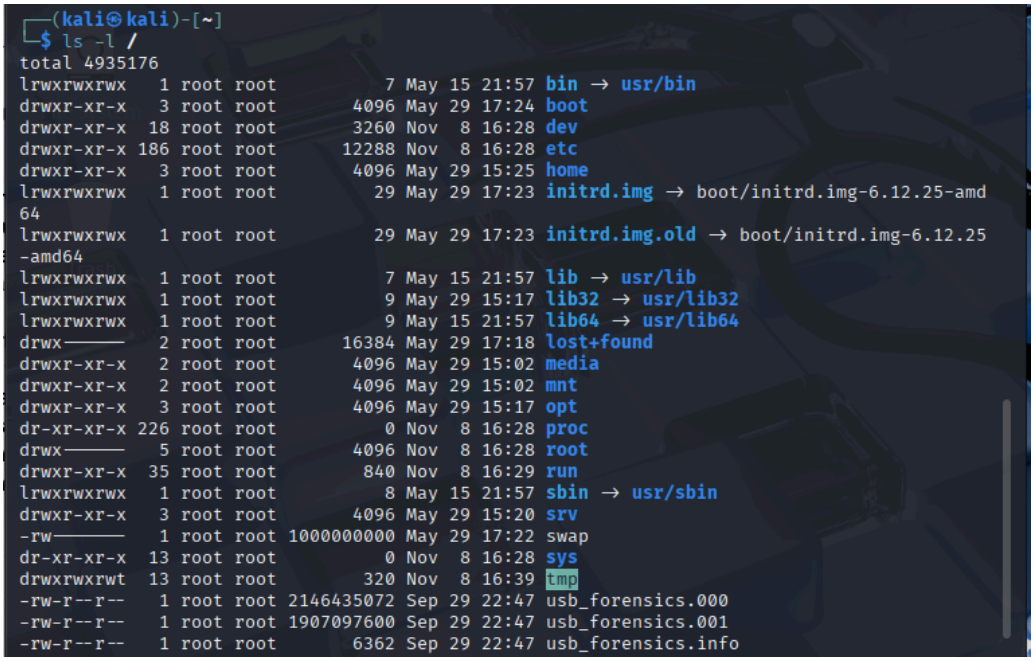
Linux Forensics Volatile Data Collection

Non-technical Summary

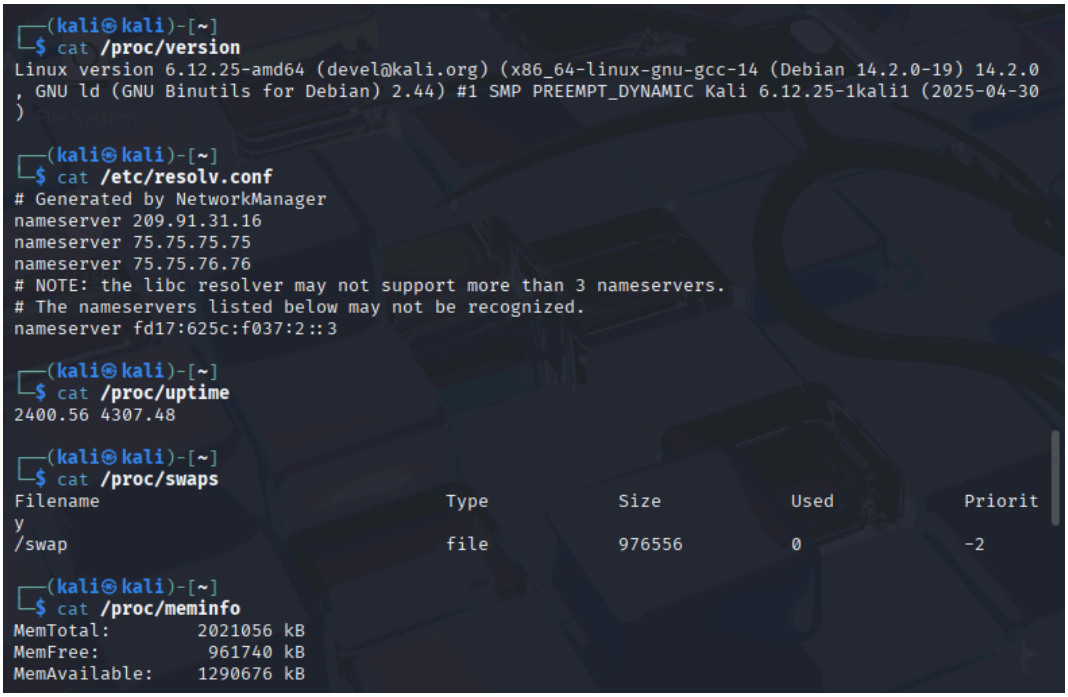
In this lab, we explored Linux file structure and important files, how to collect basic volatile information, the Linux firewall, SSH service, and port scanning with nmap. This content is important because it helps investigators understand where to locate logs for data recovery, and how we can modify it. Volatile data collection on Linux systems allows us to locate data that may have been lost or compromised. This is extremely useful to investigators for incident response. It helps them understand parts of the system such as any applications that are running, open files, modified files, or users that are logged on. These contents can be saved and be used as evidence before a system is rebooted. Linux firewall, SSH service, and using nmap for port scanning allow us to explore networks within a system. Nmap allows us to investigate weaknesses within a network. It also helps to gather information about hosts and services which is useful in auditing and pentesting.

Technical Report

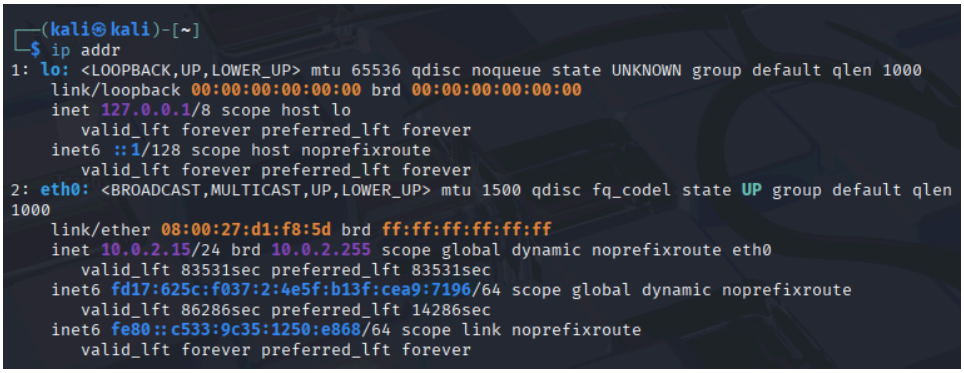
Using the ls -l command, we can see below a list of everything in the root directory. This lists the file, permissions (read, write, execute), owners (root), file size, date and time last modified, and directory. When something has a pointer -> it is a symbolic link to something else, for example, bin -> usr/bin. We also see opt and proc, which contain folders and files that are important to the operating system.



The following commands help us access important files stored by Kali. /proc/version displays the version of the kernel, gcc, and date it was compiled. /etc/resolv.conf is a nameserver configuration file. Seeing servers such as 8.8.8.8 for Google and 1.1.1.1 for cloudflare, we know there is a much more secure connection established. These servers are encrypted so that we can't see what website a user is going to. If we are within the same network, we risk being profiled, or reconnaissance can be collected on us.



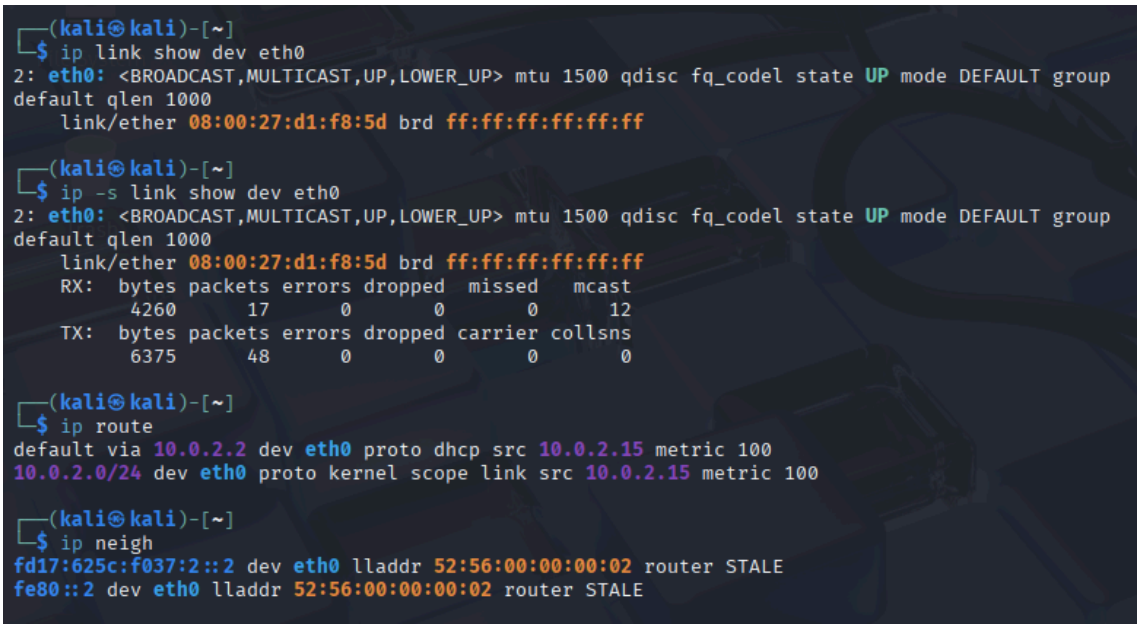
We can also see the uptime of the system, and how long all processors have been idle. /proc/swaps shows us information about swap space in the system, and /proc/meminfo shows us information about the physical memory. Swaps are used when RAM is full.



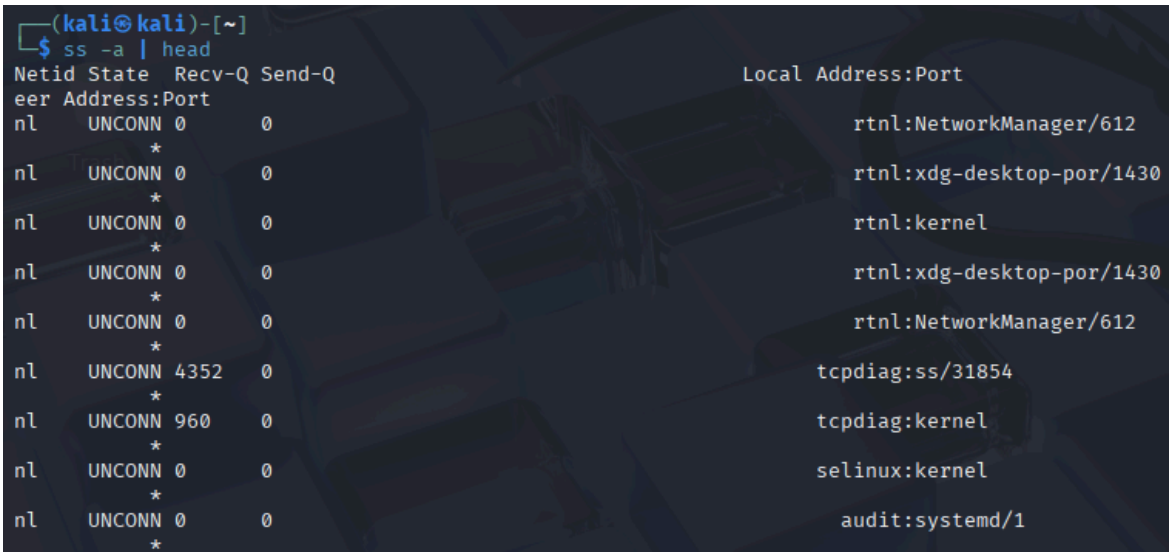
As shown above, ip addr displays the addresses of all interfaces and their information. It shows all IP addresses and MAC addresses which help with network analysis. In regards to digital forensics, this command helps to show what networks

systems had connections to, and shows specific communications between servers and IP addresses.

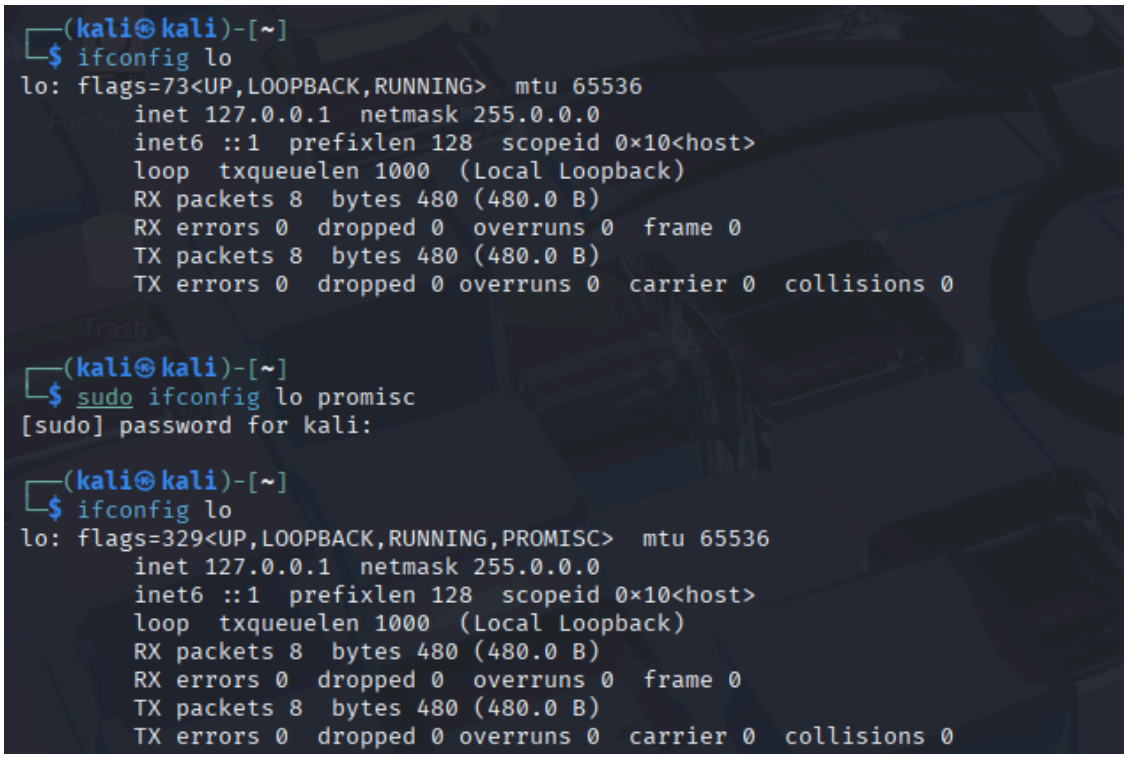
Below are additional commands that help establish information for specific interfaces, their statistics, routing table, and neighbors. These commands can help investigators understand a system's network activity, map network connections, as well as identify suspicious activity. These are especially useful when trying to identify other networks that may have been compromised in an attack.



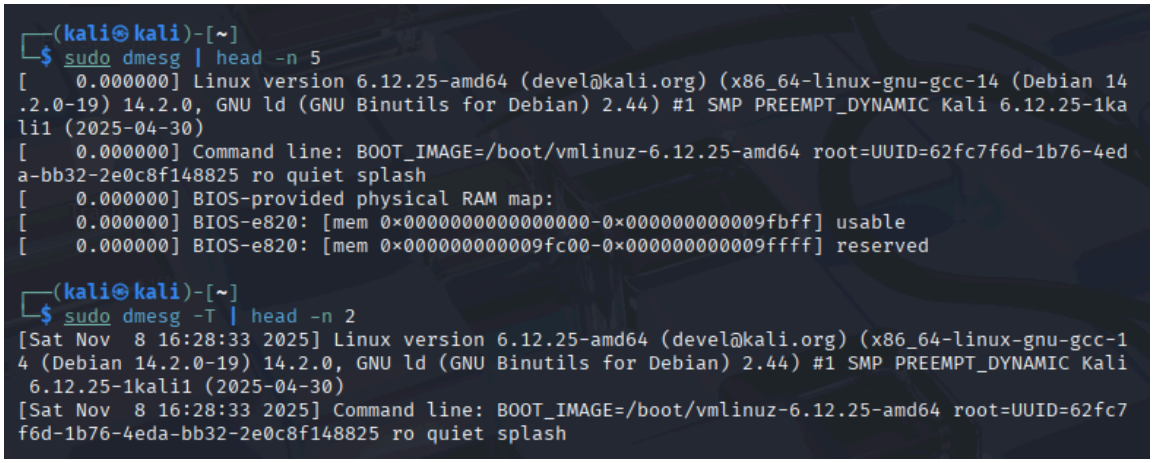
Below we see the ss -a command, which displays socket statistics. We can see that netid, connection state, received queue, send queue, and the local IP address and the port. This information helps investigators determine past and present network history.



The command ifconfig is used to check whether or not an interface is running in promiscuous mode. Promiscuous mode allows a network to accept all packets it receives. If an interface is not in promiscuous mode, we can set it using ifconfig lo promisc, and enter the system password. We see that once this command is run, after we run ifconfig lo again, we see the interface is now running promiscuous mode, shown by the PROMISC flag. Promiscuous mode helps to explore traffic on a network, and can reveal suspicious activity through network monitoring.



The command dmesg is used to get messages from the Kernel by displaying messages since the Kernel was started. This helps investigators understand all actions performed on a system. Using -T displays the output in human readable format, we can see this in the timestamp below.



Using the command lsof lists all the open files, using -i displays files opened by network connections, and -u displays files opened by a specified user with the count of returned lines.

```
┌──(kali㉿kali)-[~]
└─$ sudo lsof | head -n 4
lsof: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
      Output information may be incomplete.
lsof: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
      Output information may be incomplete.
COMMAND     PID  TID TASKCMD         USER  FD      TYPE             DEVICE  SIZE/OFF       NODE
 NAME
systemd       1                     root cwd       DIR                8,1      4096          2
 /
systemd       1                     root rtd       DIR                8,1      4096          2
 /
systemd       1                     root txt       REG                8,1    133504    2012105
 /usr/lib/systemd/systemd

┌──(kali㉿kali)-[~]
└─$ sudo lsof -i
COMMAND    PID USER FD   TYPE DEVICE SIZE/OFF NODE NAME
NetworkMa 612 root 26u  IPv4   6506      0t0  UDP 10.0.2.15:bootpc→10.0.2.2:bootps

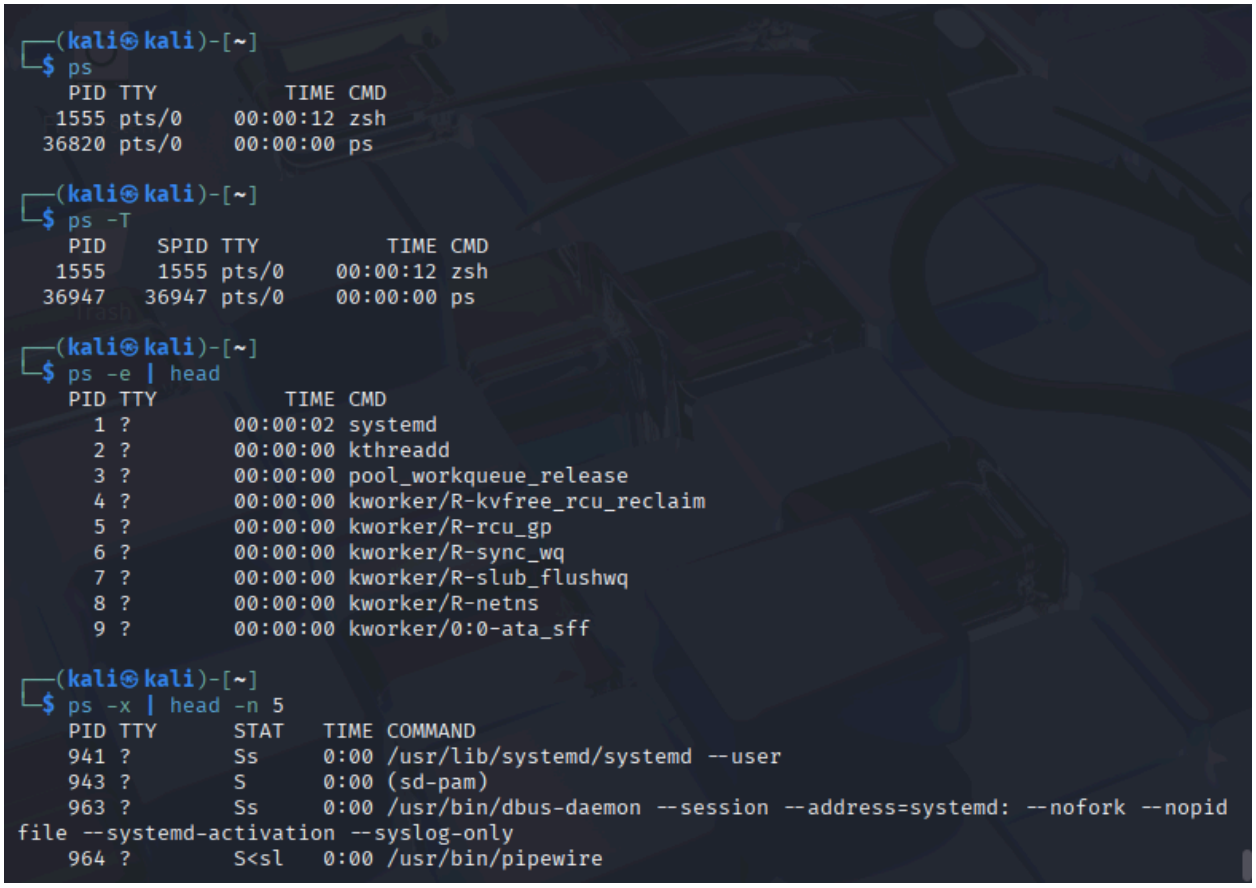
┌──(kali㉿kali)-[~]
└─$ sudo lsof -u kali | wc -l
lsof: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
      Output information may be incomplete.
lsof: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
      Output information may be incomplete.
4986
```

We can then display the first 5 lines using head -n 5. Using -c ssh displays all files opened by SSH, which is the specified process in this case. We can also explore mounted file systems by using the mount command. The df command displays the amount of space on mounted systems.

```
┌──(kali㉿kali)-[~]
└─$ sudo lsof -u kali | head -n 5
lsof: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
      Output information may be incomplete.
lsof: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
      Output information may be incomplete.
COMMAND     PID USER  FD      TYPE             DEVICE  SIZE/OFF       NODE NAME
systemd     941 kali cwd       DIR                8,1      4096          2 /
systemd     941 kali rtd       DIR                8,1      4096          2 /
systemd     941 kali txt       REG                8,1    133504    2012105 /usr/lib/systemd/sys
temd
systemd     941 kali mem       REG                8,1    117352    2363388 /usr/lib/x86_64-linu
x-gnu/libelf-0.192.so

┌──(kali㉿kali)-[~]
└─$ sudo lsof -c ssh
lsof: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
      Output information may be incomplete.
lsof: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
      Output information may be incomplete.
COMMAND    PID USER  FD   TYPE             DEVICE SIZE/OFF    NODE NAME
ssh-agent 1128 kali cwd    DIR                8,1     4096       2 /
ssh-agent 1128 kali rtd    DIR                8,1     4096       2 /
ssh-agent 1128 kali txt    REG                8,1   420224 1228916 /usr/bin/ssh-agent
ssh-agent 1128 kali mem    REG                8,1   141224 2368497 /usr/lib/x86_64-linux-gnu/os
sl-modules/legacy.so
ssh-agent 1128 kali mem    REG                8,1   825336 2364144 /usr/lib/x86_64-linux-gnu/li
bzstd.so.1.5.7
ssh-agent 1128 kali mem    REG                8,1   125376 2363786 /usr/lib/x86_64-linux-gnu/li
bz.so.1.3.1
ssh-agent 1128 kali mem    REG                8,1  2003408 2364879 /usr/lib/x86_64-linux-gnu/li
bc.so.6
```

Using the ps command allows us to get information about processes. It outputs PID, terminal type, and command line. Using -T puts this in human readable format. Using ps -e lists all running processes, and ps -x lists all processes associated with a specific user. We can specify processes by process ID, user name, session ID, arguments.



The last part of the lab explored Linux firewall, SSH service, and using nmap. Nmap is a network mapper used for port scanning. Its output displays port, state, and service. Using the command sudo ufw status verbose, we can display the details of a firewall status. To allow certain protocols, we can use sudo ufw allow to add rules. In order to communicate locally, you need a port to be open, service on.

Due to low storage on my device, I was unable to get a second Linux VM running to test open and closed ports. Using nmap in this lab allowed us to check the status of ports set by the firewall. We used OpenSSH to establish a connection between VM1 and VM2, and test the connection.

References

1. Lab 10 pdf